

Web Core View

Modernització de la interfície web d'un NVR professional

Jan Rosell Caba

Director: Fernando Gallego | *Ponent:* Carles Farré

Juny 2026



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



LANACCESS

Discover the power of video.

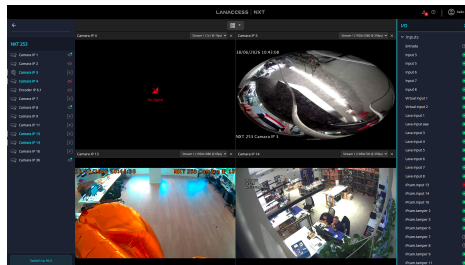
Lanaccess i els Gravadors NVR

L'empresa

- Lanaccess Telecom, SA: videovigilància professional.
- Producte estrella: família de gravadors **NXT**.
- Clients: empreses, institucions públiques, infraestructura crítica.

El dispositiu

- Fins a **152** càmeres per unitat.
- SoC ARM (NXP i.MX), Linux Yocto.
- Desplegament *standalone*: **sense Internet**.



El Problema: La Fi d'una Era

L'herència: ActiveX (anys 90)

- Plag dependent d'**Internet Explorer 11**.
 - Microsoft tanca IE el **Juny 2022**.
 - Incompatible amb macOS, Linux i mòbils.
 - Vectors de seguretat documentats (OWASP).
-
- Interfície estàtica: **sense reactivitat** ni temps real.
 - Cada versió de firmware requeria canvis al frontend.

La pressió del mercat

Clients exigien accés des de qualsevol navegador i dispositiu modern.

L'amenaça competitiva

Competidors amb interfícies web natives guanyaven terreny al catàleg NXT.

L'Objectiu: Un Sistema Web Autònom



100% Web

Cap plugin, cap instal·lació de client.

Autònom

L'NVR serveix l'app i el vídeo. Sense núvol.

Escalable

Nous firmwares no requereixen canvis al frontend.

Els Actors del Sistema

Administrador

- Configuració completa.
- Gestió d'usuaris.
- Actualitzacions de firmware.

Operador

- Vídeo en directe.
- Gravacions i timeline.
- Gestió d'alarmes.

Sistema NVR

- Emet esdeveniments push.
- Dicta l'esquema JSON.
- Gestiona els *config locks*.

6 casos d'ús principals: Autenticació · Vídeo directe · Gravacions · Alarmes · Configuració dinàmica · Administració

Requisits Crítics: Rendiment i Vídeo

RNF-01: Latència de vídeo

- Objectiu: < 300 ms de latència mitjana.
- P95: < 350 ms.
- Motiu: sales de monitorització activa.

RNF-02: Fallback automàtic

Si UDP bloquejat: commuta a HLS en ≤ 5 segons.

RNF-03: Eficiència de CPU

- Zero *polling* HTTP.
- Actualitzacions granulars (*Signals*).
- NGINX $\leq 5\%$ overhead adicional.

RNF-04: WebSocket resilient

Reconexió automàtica en ≤ 30 segons (backoff exponencial).

Requisits Crítics: Seguretat i Privadesa

Seguretat (OWASP Top 10)

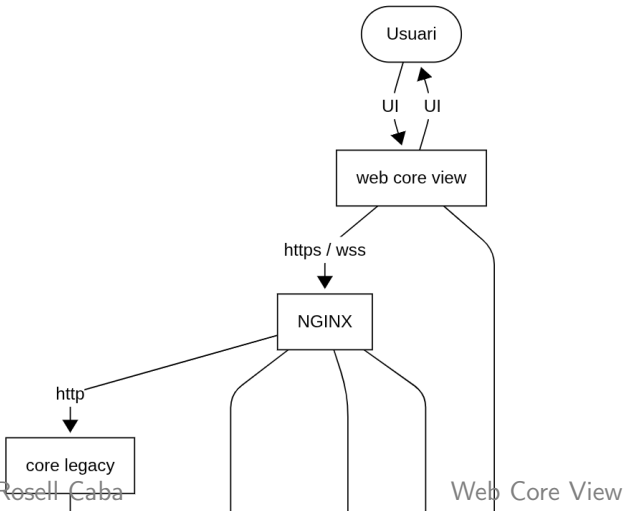
- Sense *LocalStorage* per a credencials.
- Cookies **HttpOnly** + **Secure** + **SameSite**.
- Xifratge **TLS 1.2/1.3** i **WSS**.
- Immunitat XSS i CSRF per disseny.

Privadesa (RGPD / LOPDGDD)

- **Privacy by Design**: dades sensibles només a RAM.
- Zero persistència al recarregar la pàgina.
- UNE-EN 62676: retenció 24h–30 dies.
- Sense emmagatzematge biometria.

La seguretat no és una capa afegida: és una **restricció de disseny** des del primer dia.

Arquitectura Global del Sistema



- **SPA:** Angular renderitza al navegador — allibera la CPU del gravador.
- **Microserveis C++:** cada procés gestiona un domini aïllat.
- **NGINX:** únic punt d'entrada, gestiona seguretat i routing.

Els Microserveis al Dispositiu

Capa de comunicació

- `la-core-legacy` — Config API, Auth.
- `la-db-engine` — SQLite via gRPC.
- `la-core-notifier` — Notificacions WSS.
- `la-core-alarm` — Alarmes WSS.

Capa de vídeo

- `la-video-engine` — Captura, disc, HLS.
- MediaMTX — WebRTC (WHEP) & HLS.
- Sistema FS2: fitxers a bloc brut.
- Trans-muxing H.265 *on the fly*.

- Tots desplegats com a **unitats systemd** sobre Yocto Linux.
- Comunicació interna per **Unix Domain Sockets** i gRPC.

NGINX: La Frontera de Seguretat

Proxy invers i unificació

- Un únic punt d'entrada: port 443 (HTTPS).
- Oculta ports interns dels microserveis.
- *Routing*: /stream/ → MediaMTX, /ws/ → notificadors.

Seguretat imposada

- TLS 1.2/1.3 — Cifratge **AESGCM**.
- Cookies **HttpOnly** + **Secure** + **SameSite=Strict**.
- Cap script JavaScript pot llegir la sessió.

Per què no JWT al
LocalStorage?

Un XSS al navegador pot robar un token del LocalStorage. Una cookie HttpOnly és **invisible** per a qualsevol script de la pàgina.

El Problema de la Concurrència en Configuració

Dos administradors editen simultàniament el mateix paràmetre de xarxa...

Sense control de concurrència

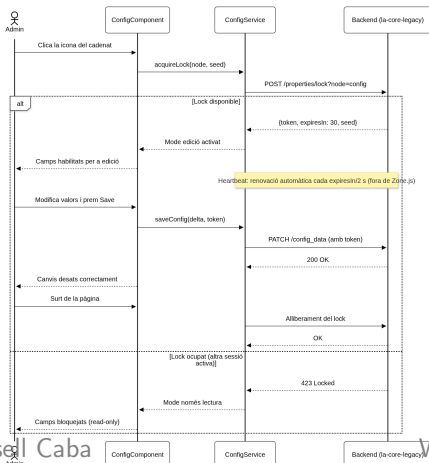
1. Admin A llegeix: IP=192.168.1.50
 2. Admin B llegeix: IP=192.168.1.50
 3. Admin A escriu: IP=10.0.0.5
 4. Admin B escriu: IP=192.168.2.99
- ! La configuració d'Admin A es perd.**

La solució: Pessimistic Lock

- Quan Admin A obre un mòdul, **adquireix el testimoni**.
- Admin B entra en **Mode Lectura** (HTTP 423).
- Cap escriptura simultània possible.

Disseny del *Config Lock* Pessimista

- **Adquisició:** POST /lock → token 32 hex.
- **Expiració:** 60–120 s sense renovació.
- **Renovació automàtica:** fora del cicle d'Angular.
- **HTTP 423 Locked** → mode lectura per a altres.



Tolerant a fallades

Talls de connexió ⇒ el cadenat caduca sol.
Cap procés queda bloquejat per sempre.

El Cor del Projecte: Motor *Schema-Driven*

El frontend no sap quantes càmeres ni quins paràmetres té el firmware. S'autoassembla en temps d'execució.

1. Backend dicta les regles

Envia un **JSON Schema** amb metadades: `_type`, `_allowedValues`, `_min`, `_max`, `_default`

⇒

2. Frontend renderitza

Angular genera automàticament *inputs*, *selects* i *checkboxes* amb validació dinàmica nativa.

Impacte empresarial directe

Nou model de maquinari o opció de firmware ⇒ **la interfície s'adapta sola**. Zero canvis al frontend.

L'Esquema JSON: De Metadades a Formulari

Metadades del Backend (exemple)

Clau	Valor
ip_address	"192.168.1.50"
_type	ipaddr"
_description	IP del dispositiu"
codec	"H264"
_type	l·list"
_allowedValues	["H264", "H265"]
max_cameras	8
_type	integer"
_min	1 _max: 152

Frontend renderitza automàticament:

- ipaddr → *input* amb regex de validació IP.
- list → `<select>` amb les opcions permeses.
- integer → *input number* amb rang [1, 152].
- boolean → *checkbox*.
- password → camp amb política de seguretat.

L'Evolució de l'Estat: D'RxJS a Angular Signals

L'antic paradigma (Zone.js + RxJS)

- Cada event revisat **tot l'arbre** de components.
- 152 càmeres $\times$ alertes constants = CPU al límit.
- Un byte de canvi $\rightarrow$ el DOM s'escrutava sencer.

El nou paradigma (Angular 19 Signals)

- Reactivitat *fine-grained*: es repinta **només** el component afectat.
- Si un relé canvia d'estat $\rightarrow$ s'actualitza **una icona**.
- Zone.js eliminat del cicle crític.

Resultat mesurat

35% de reducció de CPU al navegador en terminals de vigilància 24/7.

Orquestració de Vídeo: WebRTC / WHEP (Directe)

Protocol WebRTC via WHEP

- Negociació SDP simplificada: HTTPS POST al /stream/<cam>/whep.
- Transport: **SRTP/UDP** + DTLS (RFC 3711).
- ICE per travessar NATs i tallafocs (RFC 8445).

- Latència mesura: **150–300 ms** en LAN.
- Fins a **4 càmeres** simultànies en mosaic.

Per què WHEP i no WebRTC classic?

El WHEP (RFC 9716) estandarditza la negociació egress: una sola crida HTTP POST en lloc de l'ordinador SDP manual de WebRTC pur. Integra perfectament amb MediaMTX.

Orquestració de Vídeo: HLS (Gravacions i Fallback)

HLS per a gravacions (Timeline)

- Transport sobre **TCP/HTTPS**: passa qualsevol tallafocs.
- Segmentos .ts servits per la-video-engine.
- Línia de temps interactiva amb seeks.
- Latència: **2–5 s** (acceptable per a playback).

HLS com a fallback de directe

- Si UDP bloquejat en **> 5 s**: commuta automàticament.
- El canvi és **transparent** per l'operador.
- Angular detecta timeout i re-negocia el protocol.

La dualitat WebRTC ↔ HLS garanteix **cobertura universal** de xarxa sense comprometre la latència en LAN.

El Repte H.265: Trans-muxing al Backend

El problema

- H.265 (HEVC): **50% menys espai** que H.264 al disc.
- Molts navegadors restringeixen la descodificació H.265.
- Les gravacions FS2 estan en blocs bruts, sense contenidor.

La solució: Trans-muxing

- Llegir bloc brut de FS2.
- Injectar capçaleres SPS/PPS/VPS *on the fly*.
- Empaquetar com a MPEG-TS per a HLS.
- **Sense recomprimir:** CPU intacta.

Trans-muxing vs Transcoding

Transcoding: descomprimir + recomprimir. Alta CPU, alta qualitat.

Trans-muxing: canviar contenidor, mantenir *bitstream*. CPU quasi zero.

Gestió d'Alarmes: WebSockets i SQLite VFS

Persistència: SQLite + VFS Personalitzat

- `remote_vfs`: driver fet a mida.
- Escriptures via Unix socket: evita concurrència al disc.
- Sectors de **512 bytes**: atòmics davant talls de corrent.
- WAL mode: garanties ACID completes.

Notificació: WebSockets Push

- Zero *polling*: connexió persistent WSS.
- Event push en < 1 s al sensor.
- Sincronització entre tots els operadors: si un arxiva, tots ho veuen.
- Reconexió automàtica amb backoff exponencial.

Validació: Latència — ActiveX vs WebRTC

RNF-01 assolit		
Protocol	Latència	Estabilitat
ActiveX + RTSP (legacy)	800–2000 ms	Baixa
WebRTC / WHEP	150–300 ms	Alta
HLS (fallback)	2–5 s	Molt alta

Millora: 5–10× menys latència que la plataforma legacy.

< 300 ms de latència amb WebRTC. < 350 ms P95.
Validat en xarxa LAN amb Intel i3-8100T com a client.

Validació: Rendiment i Usabilitat

Eficiència de CPU (Signals)

- **35% de reducció** de CPU al client.
- Zero fuites de memòria (streaming 2h validat).
- Rang estable: 45–65 MB de RAM.
- Neteja correcta del `RTCPeerConnection`.

Test d'Usabilitat SUS

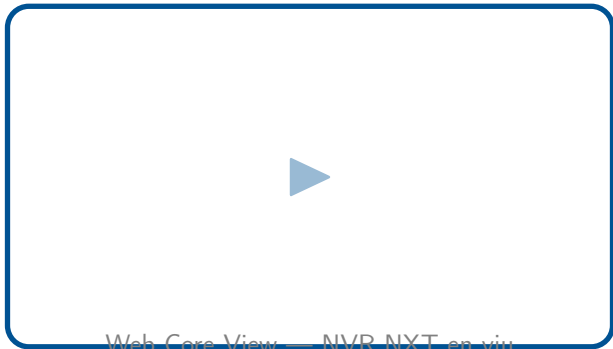
- **Puntuació: 85.6 / 100** (categoria *Excel·lent*).
- 8 participants: tècnics i operadors reals.
- Benchmark: $>80 = \text{Bo}$, $>70 = \text{Acceptable}$.
- Firmwares actualitzats un **40% més ràpid**.

Tots els RNF tècnics i d'usabilitat: assolits

WS reconnecta en ≤ 30 s · 25 connexions concurrents · zero corrupció VFS en talls de corrent

Demostració del Sistema

Vídeo de 5 minuts: el sistema en funcionament real



Planificació: Desviacions i Gestió del Risc

Fase	Previst	Executat
Gestió i Formació	40 h	40 h
Auth & Core	60 h	60 h
Manteniment Base	70 h	70 h
Alarmes & Vídeo	120 h	145 h
Gravacions & Config	170 h	190 h
Finalització	80 h	80 h
Total	540 h	585 h

Causes de la desviació (+45 h):

- H.265: SPS/PPS/VPS injection → +25 h.
- Motor Schema-Driven → +20 h.

Resultat

Desviació absorbida dins del **10% de contingència** previst. Cap feina tallada.

Pressupost: Cost Previst vs Executat

Partida	Previst	Executat	Estalvi net
Personal (hores)	13.567,50 €	14.701,50 €	358,75 € per sota del pressupost. La contingència s'ha usat parcialment (76%), no al 100%.
Amortització HW	210,00 €	210,00 €	
Costos indirectes	1.150,00 €	1.150,00 €	
Contingència (10%)	1.492,75 €	1.134,00 €	
Total	16.420,25 €	16.061,50 €	Perfil de cost >90% és cost de personal. El HW és amortitzat de maquinari de desenvolupament existent.

Sostenibilitat i Impacte Social

Ambiental

- 35% menys CPU → menys consum elèctric.
- Gestió remota → menys desplaçaments de tècnics.
- Hardware existent reutilitzat.

Social

- UI clara redueix estrès en torns 24/7.
- Accessibilitat multiplataforma (Linux, Mac).
- Privadesa per disseny (RGPD).

Econòmica

- Schema-driven: zero cost per nou firmware.
- PWA futura: diagnòstic offline en mòbils.
- Vida útil prolongada del producte NXT.

No es guarda cap dada biomètrica. Cap decisió automatitzada sense supervisió humana. Les dades d'imatge mai surten de la xarxa local.

Conclusions: Objectius Assolits

Objectiu	Resultat
Plataforma 100% web (sense plugins)	✓
Latència vídeo < 300 ms	✓
Motor Schema-Driven	✓
Seguretat OWASP Top 10	✓
Reducció de CPU (35%)	✓
Desplegament autònom (standalone)	✓
Pressupost i terminis	✓

Línies de futur

- Dashboard de reports en temps real.
- *Snapshot* automàtic en alarma.
- App mòbil (PWA / Ionic).
- Anàlisi de vídeo amb IA (bounding boxes).

Moltes gràcies

Quedo a la disposició del tribunal per a qualsevol pregunta

Autor: Jan Rosell Caba

Director: Fernando Gallego

Ponent: Carles Farré

Latència WebRTC: 150–300 ms

Reducció CPU: 35%

SUS: 85.6 / 100



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



LANACCESS

Discover the power of video.